

FIAP — Faculdade de Informática e Administração Paulista

LYNN

Plataforma de Inteligência Conversacional B2B
Ecossistema TOTVS — Modelo Task as a Service (TaaS)

Domain Driven Design — Java

Caio N. Battista - RM: 561383

Lucas Cavalcante - RM: 562857

Matheus Rodrigues - RM: 561689

Manoah Leão - RM: 563713

Jean Pierre - RM: 566534

São Paulo — Junho de 2026

Sumário

- 1.Descritivo do Projeto
 - 1.1Contexto e Problema
 - 1.2Objetivo da Solução
 - 1.3Justificativa Técnica
 - 1.4Funcionalidades Implementadas
- 2.Diagrama de Classes UML
- 3.Descrição das Classes e Relacionamentos
 - 3.1Enumerações
 - 3.2Classes de Modelo
 - 3.3Classes de Serviço

Descritivo do Produto

1.1 Contexto e Problema

No ecossistema corporativo B2B, milhares de reuniões comerciais, de suporte e de implantação são realizadas diariamente entre equipes internas da TOTVS e seus clientes. Essas interações geram um volume massivo de informações estratégicas — dores operacionais, menções a concorrentes, relatos de instabilidades técnicas e oportunidades de expansão de contrato — que, na prática, ficam aprisionadas em transcrições de texto não estruturadas e nunca são aproveitadas de forma sistemática. A ausência de um mecanismo inteligente para processar e classificar essas informações representa uma perda significativa de receita potencial e uma vulnerabilidade na retenção de clientes, pois sinais críticos de churn e insatisfação passam despercebidos pelas lideranças estratégicas.

1.2 Objetivo da Solução

A plataforma **LYNN** foi concebida como um motor de inteligência conversacional que transforma transcrições desestruturadas de reuniões em **insights acionáveis** para os setores Comercial, Sucesso do Cliente (CS) e Qualidade/Produto. O sistema recebe o texto bruto de uma conversa, enriquece-o com metadados contextuais (quem enviou, qual cliente, qual produto TOTVS está em uso) e aplica um motor de análise semântica que classifica automaticamente o conteúdo em cinco categorias de insight: oportunidades de **Upsell**, **Cross-sell**, risco de **Churn**, relatos de **Bug de Sistema** e **Fricção de Usabilidade (UX)**. Para cada insight, o motor calcula um nível de risco (Baixo, Médio, Alto ou Crítico), extrai o trecho exato da transcrição que motivou a classificação e gera uma recomendação de ação direcionada ao produto TOTVS do contexto.

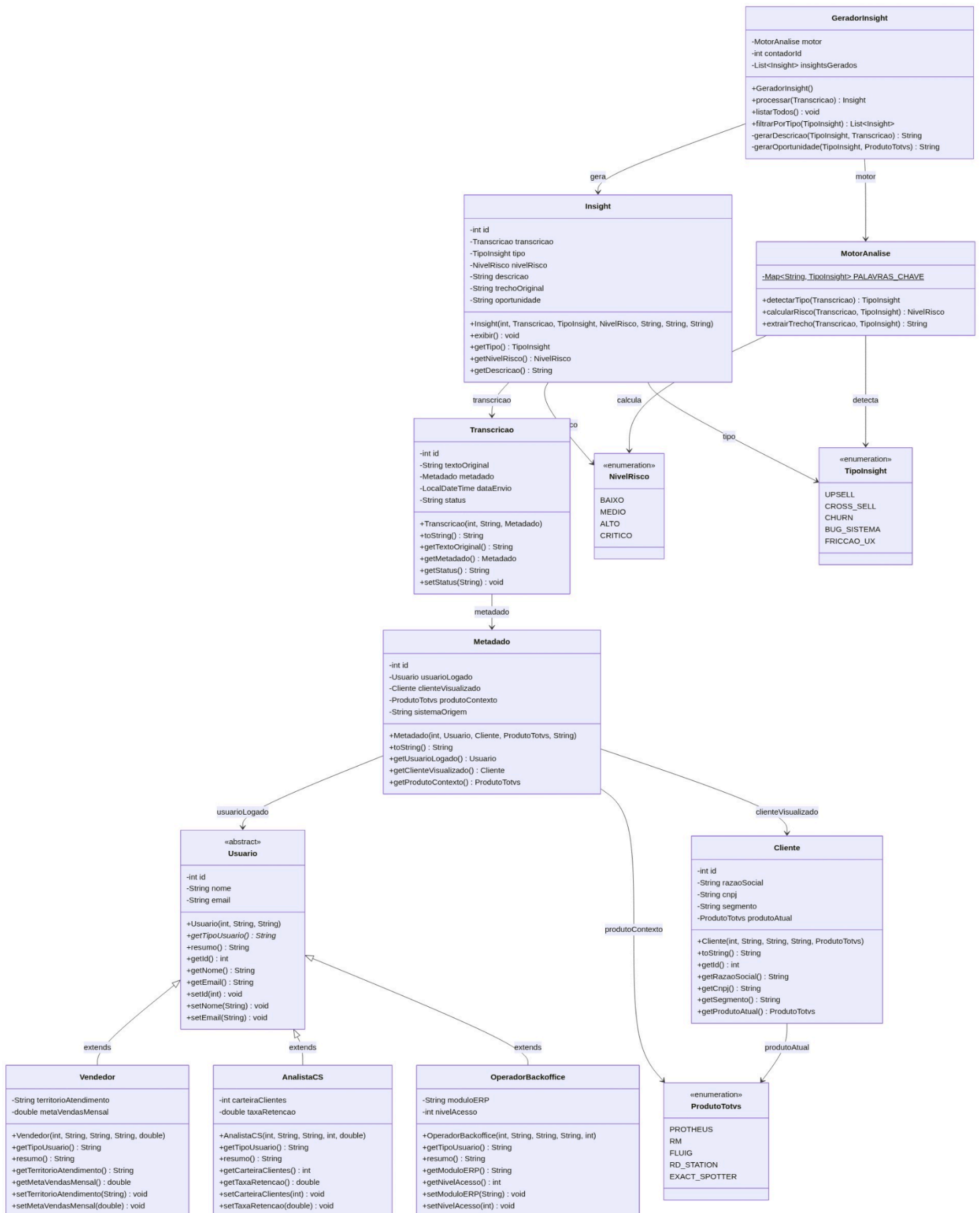
1.3 Justificativa Técnica

O projeto foi desenvolvido em **Java 21** com arquitetura orientada a objetos seguindo os princípios de Domain Driven Design. A escolha por Java puro (sem frameworks como Spring) foi intencional para demonstrar domínio dos fundamentos de OO — herança, polimorfismo, encapsulamento e composição — sem depender de abstrações de terceiros. A estrutura de pacotes segue a separação de responsabilidades em três camadas: **enums** (constantes de domínio), **model** (entidades de negócio) e **service** (lógica de processamento). A classe abstrata **Usuário** serve como base polimórfica para os três perfis de persona da solução (**Vendedor**, **AnalistaCS** e **OperadorBackoffice**), cada um com atributos específicos de seu contexto operacional. O motor de análise utiliza um mapa de palavras-chave ponderadas para detectar padrões semânticos no texto e classificar o tipo de insight, simulando o comportamento de um pipeline de NLP simplificado.

1.4 Funcionalidades Implementadas

O sistema oferece as seguintes funcionalidades através de uma interface console interativa:

(1) Cadastro de Usuários com suporte a três perfis distintos (Vendedor com território e meta, Analista CS com carteira e taxa de retenção, Operador Backoffice com módulo ERP e nível de acesso); **(2) Cadastro de Clientes** com razão social, CNPJ, segmento e produto TOTVS atual; **(3) Envio de Transcrições** vinculando texto bruto a um par usuário-cliente com enriquecimento automático de metadados e processamento imediato pelo motor de análise; **(4) Listagem de Insights** gerados com exibição de tipo, nível de risco, descrição, trecho original e oportunidade recomendada; e **(5) Filtro de Insights por Tipo**, permitindo segmentar a visualização por categoria (Upsell, Cross-sell, Churn, Bug ou Fricção UX). O sistema carrega dados de exemplo no início da execução para demonstrar o fluxo completo de ponta a ponta.



2. Diagrama de classes UML

Descrição das Classes e Relacionamentos

3.1 Enumerações

Enum	Valores	Propósito
<code>NivelRisco</code>	BAIXO, MEDIO, ALTO, CRITICO	Classifica a severidade de um insight gerado, permitindo priorização de ações.
<code>ProdutoTotvs</code>	PROTHEUS, RM, FLUIG, RD_STATION, EXACT_SPOTTER	Representa os produtos do ecossistema TOTVS que o cliente pode utilizar.
<code>TipoInsight</code>	UPSELL, CROSS_SELL, CHURN, BUG_SISTEMA, FRICCAO_UX	Categoriza o tipo de inteligência extraída da transcrição.

3.2 Classes de Modelo

Classe	Tipo	Atributos Principais	Descrição
<code>Usuario</code>	Abstrata	<code>id</code> , <code>nome</code> , <code>email</code>	Classe base para todos os perfis de usuário do sistema. Define os métodos abstratos <code>getTipoUsuario()</code> e o método polimórfico <code>resumo()</code> .
<code>Vendedor</code>	Concreta	<code>territorioAtendimento</code> , <code>metaVendasMensal</code>	Especialista de vendas (Persona 1). Herda de <code>Usuario</code> . Atua em prospecção via RD Station CRM.
<code>AnalistaCS</code>	Concreta	<code>carteiraClientes</code> , <code>taxaRetencao</code>	Analista de Customer Success (Persona 2). Herda de <code>Usuario</code> . Conduz reuniões de retenção e implantação.
<code>OperadorBackoffice</code>	Concreta	<code>moduloERP</code> , <code>nivelAcesso</code>	Operador de backoffice (Persona 3). Herda de

			Usuario. Utiliza Core ERP (Protheus, RM).
Cliente	Concreta	<code>razaoSocial, cnpj, segmento, produtoAtual</code>	Representa o cliente B2B da TOTVS. Associado a um produto do ecossistema via enum ProdutoTotvs .
Metadado	Concreta	<code>usuarioLogado, clienteVisualizado, produtoContexto, sistemaOrigem</code>	Empacota o contexto da transcrição: quem enviou, sobre qual cliente, em qual produto e de qual sistema de origem.
Transcricao	Concreta	<code>textoOriginal, metadado, dataEnvio, status</code>	Texto bruto da reunião com metadados contextuais. O status transiciona de AGUARDANDO_ANALISE → EM_ANALISE → CONCLUIDO.
Insight	Concreta	<code>tipo, nivelRisco, descricao, trechoOriginal, oportunidade</code>	Resultado do processamento. Contém a classificação, o risco, o trecho relevante e a recomendação de ação.

3.3 Classes de Serviço

Classe	Métodos Principais	Descrição
MotorAnalise	<code>detectarTipo(), calcularRisco(), extrairTrecho()</code>	Núcleo de processamento semântico. Utiliza um mapa de palavras-chave ponderadas para identificar padrões no texto, calcular o nível de risco com base na frequência de ocorrências e extrair o trecho relevante da transcrição.
GeradorInsight	<code>processar(), filtrarPorTipo(), listarTodos()</code>	Orquestra o fluxo de análise. Recebe a transcrição, coordena o MotorAnalise para detectar tipo e risco, gera descrições e oportunidades contextualizadas, e armazena os insights produzidos.

3.4 Relacionamentos

Relacionamento	Tipo	Descrição
Usuario → Vendedor, AnalistaCS, OperadorBackoffice	Herança	Três subclasses concretas estendem a classe abstrata, cada uma com atributos específicos de seu perfil.
Metadado → Usuario, Cliente	Associação	Metadado referencia o usuário logado e o cliente visualizado no momento do envio.
Transcricao → Metadado	Composição	Cada transcrição possui obrigatoriamente um metadado que contextualiza a interação.
Insight → Transcricao	Associação	Cada insight é derivado de uma transcrição específica, mantendo rastreabilidade.
GeradorInsight → MotorAnalise	Composição	O gerador delega a análise ao motor, seguindo o princípio de responsabilidade única